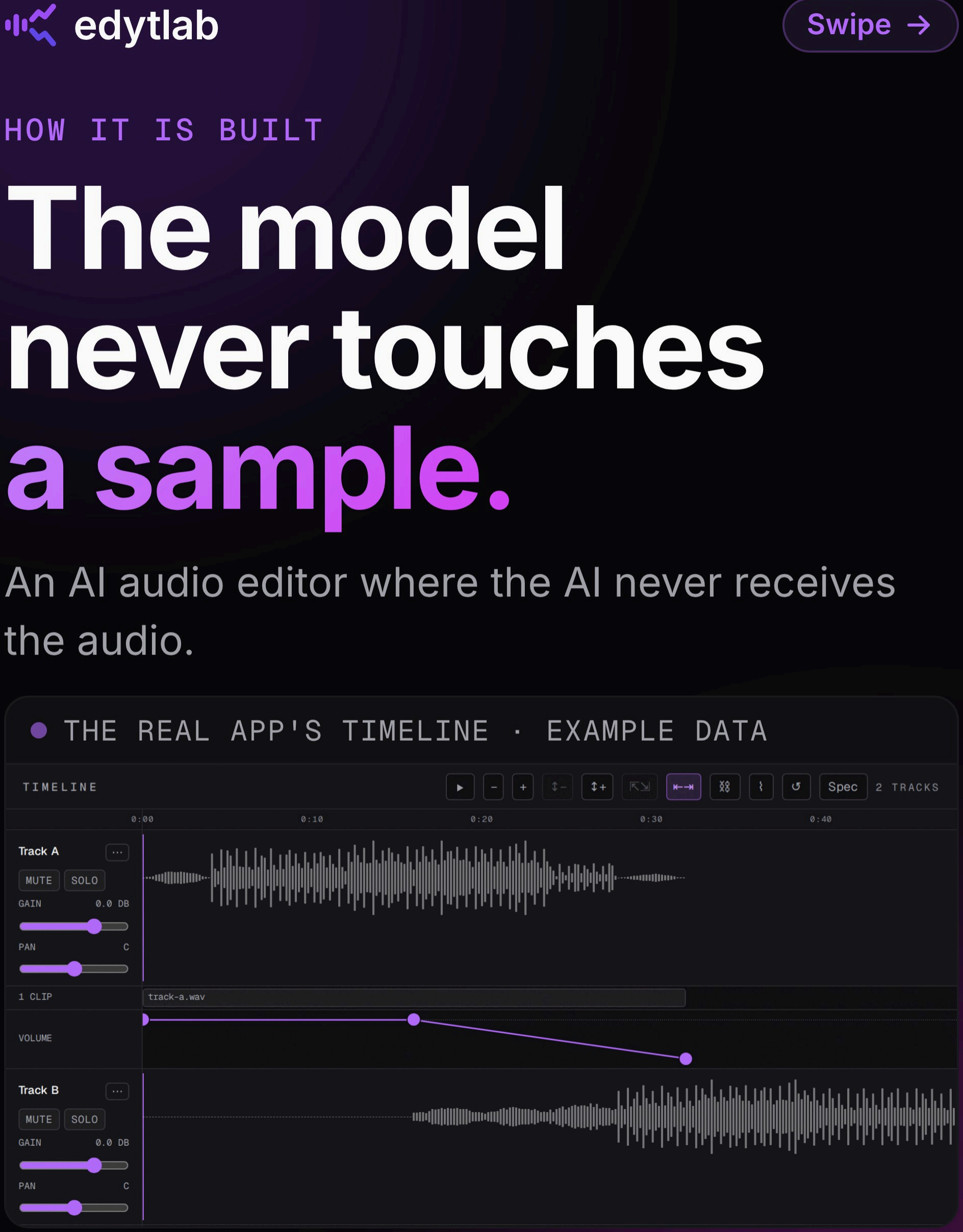
edytlab

Swipe →

HOW IT IS BUILT

The model never touches a sample.

An AI audio editor where the AI never receives the audio.



The agent chooses.
Tools do the work.

Match B's tempo to A and crossfade them.

↓ text about the session

The model — your provider, or Ollama

Reads text: tempo, key, loudness. Answers with tool calls. Never receives the audio.

↓ tool calls

ON YOUR MACHINE

Deterministic tools

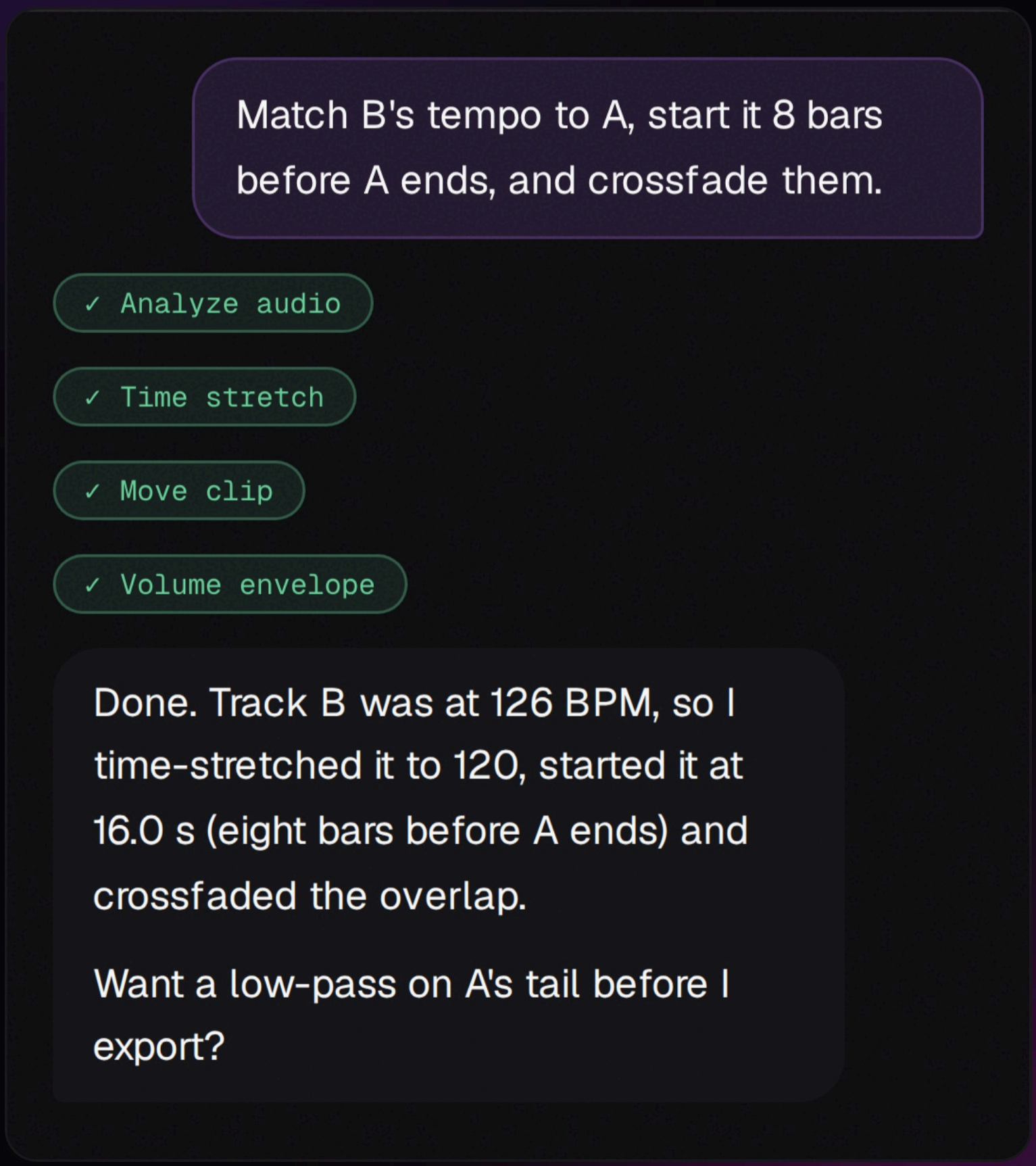
Fade, EQ, time-stretch, loudness, export. Rust functions, no model in the audio path.

↓ edits

Your audio

Decoded, edited, mixed and exported here, in the Rust engine.

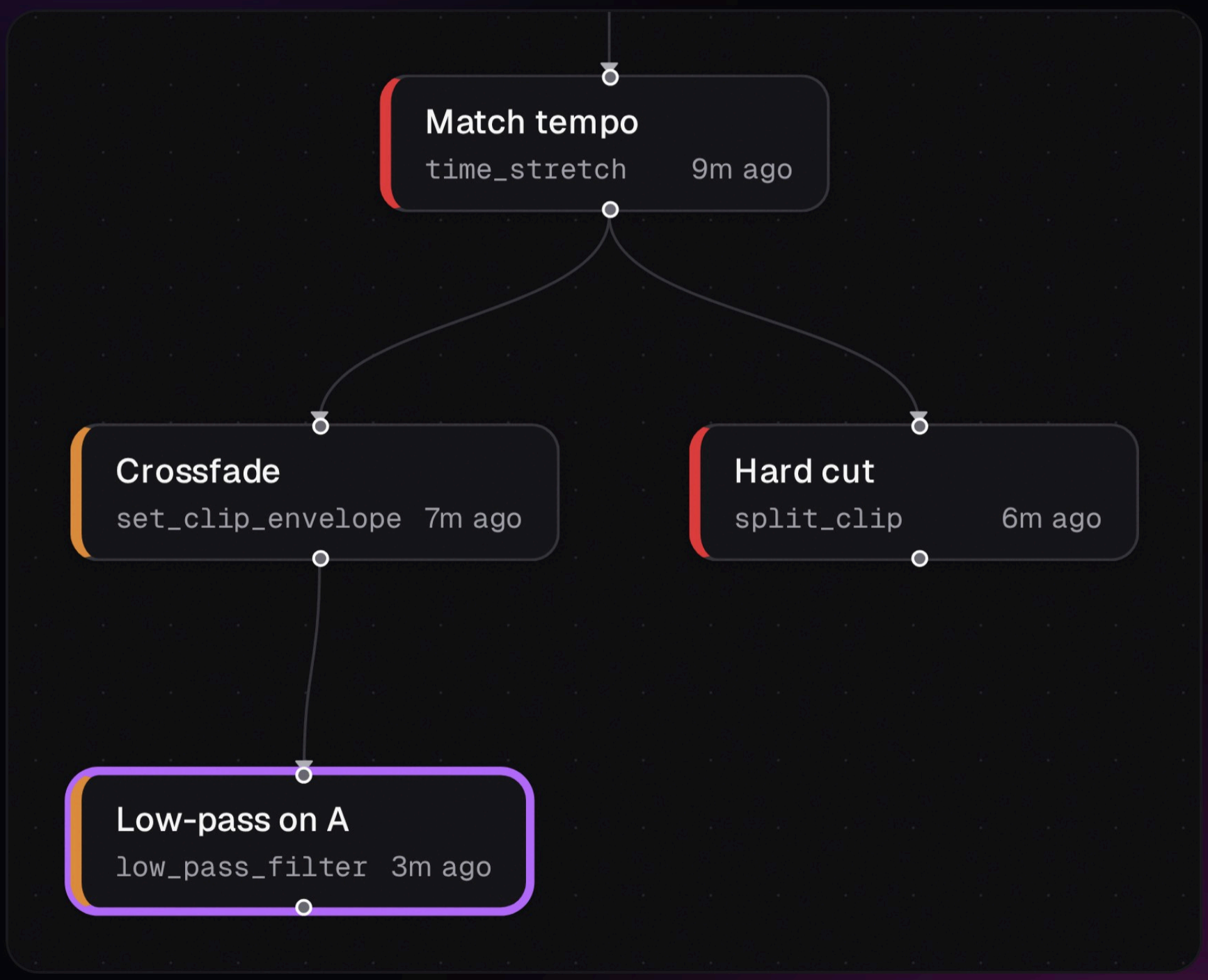
Every change is a tool call.



The real app's chat, with a scripted example conversation

Every edit is a node.

Fork a take, A/B it against the original, go back. Undo follows the path you actually took.

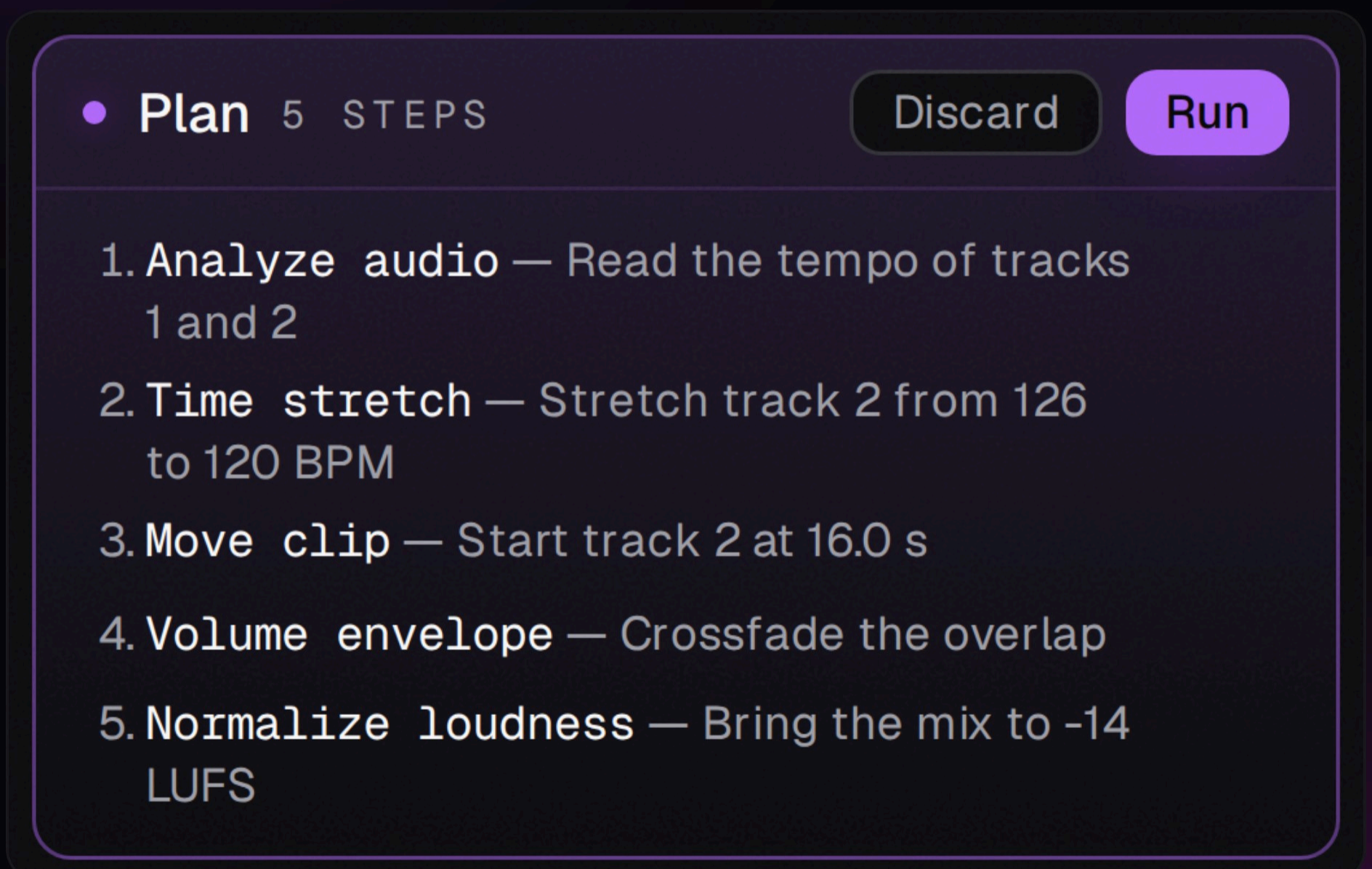


The real app's history graph, with example data

Right-click a node to compare it with another, or to set it as the head.

Plan first, if you want it.

Turn it on and the agent's steps wait for you. No edit runs until you press Run.



The real app's plan card, with a scripted example plan

Discard it and nothing runs. Reword a step and the agent proposes again.

Time-stretch, no C++ library.

Rubber Band was the plan. It needs a different native package on each of three CI targets, the kind of dependency that breaks every build at once. So the phase vocoder is written here, in pure Rust.



What it buys

The same build on macOS, Windows and Linux.

What it costs

Dense material can sound smeared at large changes.

The reasoning is in the module docs of `crates/audio-time/src/vocoder.rs`

Read the source.

MIT-licensed. A Tauri 2 shell, a React front end and a Rust workspace behind it.

---

0 bytes
of your audio uploaded

Any AI provider
one runs locally

Dozens of tools
the agent can call

github.com/laadtushar/edytlab

What would you want to review first?